Machine learning
- Linear regression

CdL Informatica e Innovazione Digitale

Valerio Freschi

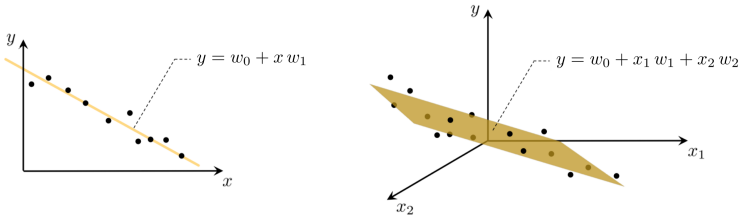
Università di Urbino - DiSPeA
valerio.freschi@uniurb.it

Linear regression

- Linear regression is essentially the task of fitting a line (a hyperplane in higher dimensions) to a set of input/output data points
- Purpose:
  - to produce a trend-line (or a trend-curve) to visualize/summarize data
  - to learn a model for predicting future values
- Data for regression problems usually takes the form:
  $(\boldsymbol{x}_1, y_1)$, $(\boldsymbol{x}_2, y_2)$, ..., $(\boldsymbol{x}_P, y_P)$ or $\{(\boldsymbol{x}_p, y_p)\}_{p=1}^{P}$
- In general each input $\boldsymbol{x}_p$ is a $N \times 1$ vector: $\boldsymbol{x}_p = \begin{bmatrix} x_{1,p} \\ x_{2,p} \\ \vdots \\ x_{N,p} \end{bmatrix}$

Linear regression

- For scalar inputs ($N = 1$) regression linear regression corresponds to fitting a line to data points in a two-dimensional space
- Indeed two parameters, the vertical intercept (w_0, *bias* weight) and the slope (w_1) have to be found s.t. $w_0 + x_p w_1 \approx y_p, \quad p = 1, ..., P$
- For N-dimensional input: $w_0 + x_{1,p} w_1 + x_{2,p} w_2 + \cdots + x_{N,p} w_N \approx y_p, \quad p = 1, ..., P$



Linear regression

- w_0 is the bias, $w_1.w_2, ..., w_N$ are called *feature-touching* weights
- We may write: $\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}$ $\mathring{\boldsymbol{x}}_p = \begin{bmatrix} 1 \\ x_{1,p} \\ x_{2,p} \\ \vdots \\ x_{N,p} \end{bmatrix}$
- A compact formulation: $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} \approx y_p \quad p = 1, ..., P$

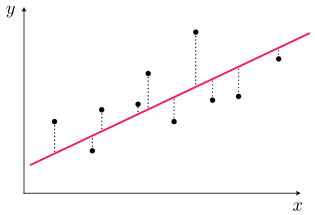
Least Squares cost function

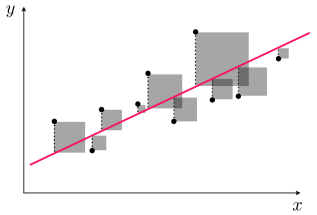
- How can we find the hyperplane parameters?
- We need to define a *cost function* to measure how well a *model* fits data
- The Least Squares cost function is one possible (very often used) choice
- We ideally want $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}$ to be as close as possible to y_p
- $g_p(\boldsymbol{w}) = \left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} - y_p\right)^2$ (pointwise cost function)
- We want all P values simultaneously too be small, hence we average over the whole dataset:

$$g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} g_p(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P}\left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} - y_p\right)^2$$

(Least Squares cost function)

Least Squares cost function





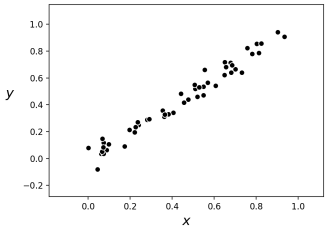
- We want to find the optimal parameter vector $\boldsymbol{w}$, i.e. the one that minimizes $g(\boldsymbol{w})$
- In other words we should solve an unconstrained optimization problem:
- $\underset{\boldsymbol{w}}{\text{minimize}} \ \frac{1}{P}\sum_{p=1}^{P}\left(\mathring{\boldsymbol{x}}_p^T\boldsymbol{w} - y_p\right)^2$

Least Squares cost function

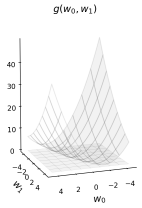
- An important result that can be demonstrated: the Least Squares cost function is convex for *any* dataset and it is infinitely differentiable
- As a consequence, we may apply any local optimization algorithm to minimize it
- Being it a quadratic function, a single Newton step can minimize it, leading to the so-called *normal equations*

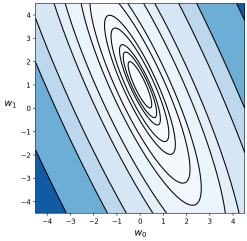
Least Squares cost function

- An example of the convexity of Least Squares:



(a)

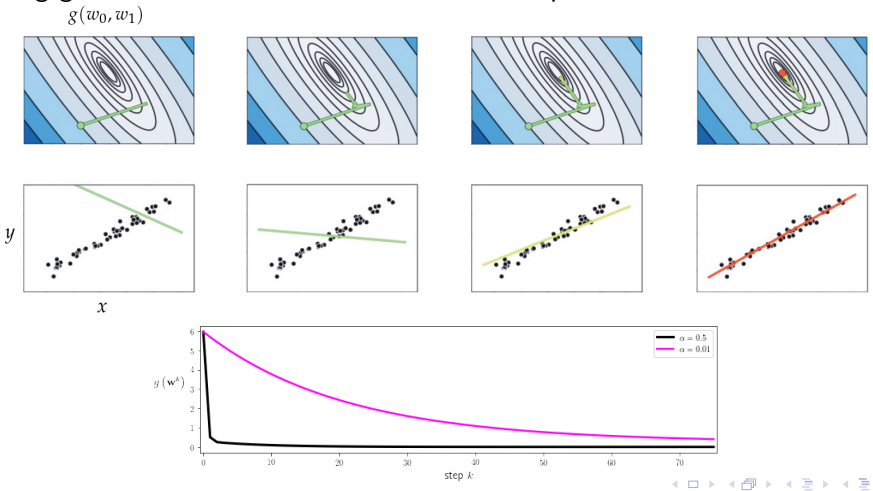




(b)

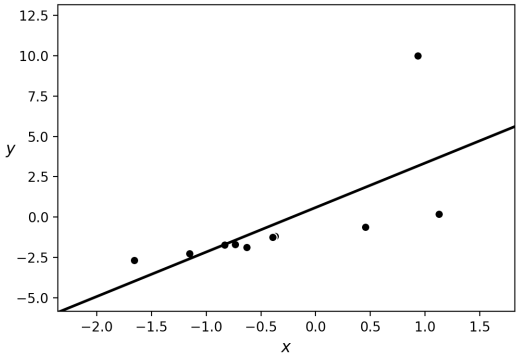
Least Squares cost function

- An example of the convexity of Least Squares
- Solving using gradient descent with $\alpha = 0.5$ for 75 steps



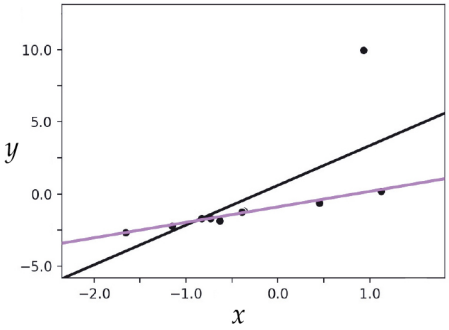
Least Absolute Deviations

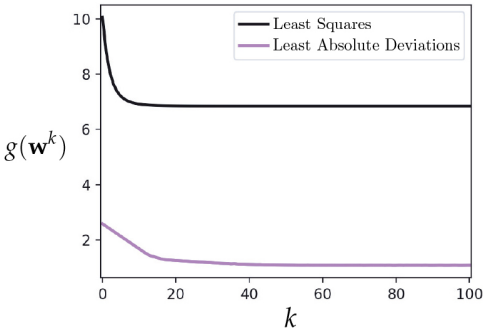
- A potential problem with Least Squares cost function is that squaring errors increases the importance of larger errors
- To overcome the sensitivity to outliers the Least Absolute Deviations cost function can be used



Least Absolute Deviations

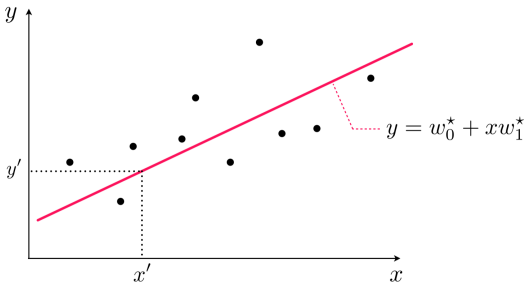
- $g_p(\boldsymbol{w}) = \left|\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} - y_p\right| \qquad p = 1, ..., P$
- $g(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} g_p(\boldsymbol{w}) = \frac{1}{P}\sum_{p=1}^{P} \left|\mathring{\boldsymbol{x}}_p^T \boldsymbol{w} - y_p\right|$





Regression and predictions

- How could we make a prediction with a trained regression model?
- How can we evaluate the quality of a model?
- Let $\boldsymbol{w}^\star$ be the optimal set of weights that minimises a regression cost function
- $\text{model}\,(\boldsymbol{x}, \boldsymbol{w}^\star) = \mathring{\boldsymbol{x}}^T \boldsymbol{w}^\star = w_0^\star + x_1 w_1^\star + x_2 w_2^\star + \cdots + x_N w_N^\star$
- Given an input $\boldsymbol{x}_0$, we predict the corresponding output y_0 as: $\text{model}\,(\boldsymbol{x}_0\,, \boldsymbol{w}^\star) = y_0$

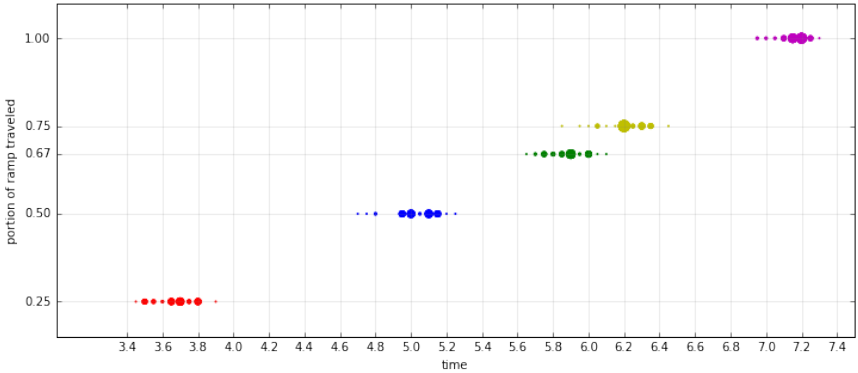


Regression and predictions

- To evaluate the quality of a trained model we introduce the Mean Squared Error (MSE)
- $\text{MSE} = \frac{1}{P} \sum_{p=1}^{P} \left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}^\star\right) - y_p\right)^2$
- To reduce the effect of outliers we may take its square root
- Analogously for the Least Absolute Deviations cost we introduce the Mean Absolute Deviations (MAD)
- $\text{MAD} = \frac{1}{P} \sum_{p=1}^{P} \left|\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}^\star\right) - y_p\right|$

Weighted regression

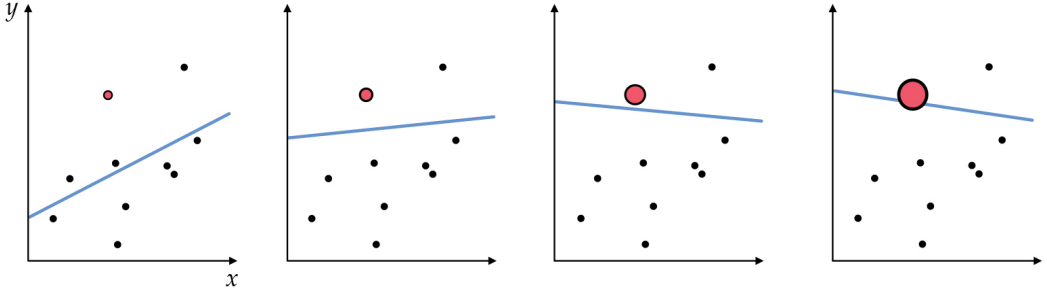
- The importance of points involved in regression can be emphasized or de-emphasized $\rightarrow$ weighted regression



- $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \beta_p \left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}\right) - y_p\right)^2$

Weighted regression

- We could also account for confidence of each data point
- If we trust a point, we set its weight β_p high



- $g(\boldsymbol{w}) = \frac{1}{P} \sum_{p=1}^{P} \beta_p \left(\text{model}\left(\boldsymbol{x}_p, \boldsymbol{w}\right) - y_p\right)^2$

Multi-output regression

- Scalar output regression covers the vast majority of practice cases
- In principle regression can be applied also when both input and output are vectors (say N-dimensional and C-dimensional)
- To keep similar notations we assume inputs $\boldsymbol{x}_p$ are column ($[N \times 1]$) vectors and outputs $\boldsymbol{y}_p$ are row ($[1 \times C]$) vectors
- Under the assumption of a linear relationship between $\boldsymbol{x}_p$ and the cth dimension of the output $y_{c,p}$: $\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_c \approx y_{c,p} \qquad p = 1, ..., P$
- Keeping the linear assumption for all the C output entries:

$$\boldsymbol{W} = \begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} & \cdots & w_{0,C-1} \\ w_{1,0} & w_{1,1} & w_{1,2} & \cdots & w_{1,C-1} \\ w_{2,0} & w_{2,1} & w_{2,2} & \cdots & w_{2,C-1} \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ w_{N,0} & w_{N,1} & w_{N,2} & \cdots & w_{N,C-1} \end{bmatrix}$$

Multi-output regression

- $\mathring{\boldsymbol{x}}_p^T \boldsymbol{W} = \begin{bmatrix} \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_0 & \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_1 & \cdots & \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_{C-1} \end{bmatrix}$
- $\mathring{\boldsymbol{x}}_p^T \boldsymbol{W} \approx \boldsymbol{y}_p \qquad p = 1, ..., P$
- Cost functions:
  - Least Squares cost function:
    $g(\boldsymbol{W}) = \frac{1}{P} \sum_{p=1}^{P} \left\| \mathring{\boldsymbol{x}}_p^T \boldsymbol{W} - \boldsymbol{y}_p \right\|_2^2 = \frac{1}{P} \sum_{p=1}^{P} \sum_{c=0}^{C-1} \left(\mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_c - y_{c,p} \right)^2$
  - Least Absolute Deviations:
    $g(\boldsymbol{W}) = \frac{1}{P} \sum_{p=1}^{P} \left\| \mathring{\boldsymbol{x}}_p^T \boldsymbol{W} - \boldsymbol{y}_p \right\|_1 = \frac{1}{P} \sum_{p=1}^{P} \sum_{c=0}^{C-1} \left| \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_c - y_{c,p} \right|$
- Property: like scalar-valued version these cost functions are always convex
- They can also be decomposed over weights $\boldsymbol{w}_c$:
  $g(\boldsymbol{W}) = \sum_{c=0}^{C-1} \left(\frac{1}{P} \sum_{p=1}^{P} \left| \mathring{\boldsymbol{x}}_p^T \boldsymbol{w}_c - y_{c,p} \right| \right) = \sum_{c=0}^{C-1} g_c(\boldsymbol{w}_c)$